

Retrieval-Augmented Generation (RAG) Systems

Research Report — Concepts, Retrieval Techniques, Real-World Case Studies and a Novel Application

1. Introduction

Retrieval-Augmented Generation (RAG) combines information retrieval with a generative language model. Instead of relying only on knowledge stored in model parameters, a RAG system retrieves relevant external information and supplies it as context before generating a response.

2. Importance of RAG

- Provides access to external and domain-specific knowledge.
- Can incorporate updated information without retraining the whole model.
- Helps ground responses in retrieved source material.
- Useful for private organizational knowledge bases.
- Can support source citations and evidence-based answers.

3. Basic RAG Architecture

- Data ingestion: collect and prepare documents, webpages, databases or other sources.
- Chunking: divide documents into retrievable passages.
- Embedding: convert passages into vector representations.
- Indexing: store vectors and metadata in a searchable index.
- Retrieval: find passages relevant to a query.
- Augmentation: add retrieved context to the model prompt.
- Generation: produce a response grounded in the retrieved context.

4. Data Retrieval Techniques

Technique	Description	Strength
Keyword / BM25	Matches query terms with document terms.	Good for exact keywords.
Dense Vector Search	Uses embeddings for semantic similarity.	Good for meaning-based queries.
Hybrid Search	Combines keyword and vector retrieval.	Balances exact and semantic matching.
Metadata Filtering	Filters by date, author, category or permissions.	Improves precision and access control.
Reranking	Reorders candidate passages by relevance.	Improves final context quality.
Knowledge Graph Retrieval	Uses entities and relationships.	Useful for connected facts.

5. Query Processing and Generation

A user query is processed and sent to a retriever. Relevant passages are selected, optionally reranked and filtered, and then inserted into a prompt. The language model generates the final answer using the retrieved evidence. Advanced systems may rewrite ambiguous queries or retrieve information in multiple stages.

6. Case Study: Customer Support

RAG can power customer-support assistants by retrieving product manuals, troubleshooting guides, policies and FAQs before generating an answer. This helps the assistant respond using organization-approved

information.

Workflow: Customer question → retrieve documentation → rerank evidence → generate grounded response → show sources or escalate when necessary.

7. Case Study: Enterprise Knowledge Search

Organizations can use RAG as a conversational interface over internal policies, technical documentation, project files and reports. Retrieval allows answers to reflect organization-specific information without requiring the language model to memorize every document.

8. Case Study: Content and Research Assistance

RAG can assist researchers and content teams by retrieving relevant reports, articles, specifications or approved reference material before generating summaries and drafts. This can improve grounding and reduce manual searching.

9. Benefits of RAG

- Better grounding in external information.
- Access to private and domain-specific knowledge.
- Easier updates to knowledge bases.
- Potential transparency through source references.
- Useful for large document collections.
- Can integrate with databases and APIs.

10. Challenges and Limitations

- Poor retrieval produces poor generation.
- Irrelevant or duplicated chunks can confuse the model.
- Large contexts increase cost and latency.
- Sources may be outdated or contradictory.
- Access-control errors can expose restricted data.
- The model may still produce unsupported claims.
- Both retrieval and generation need evaluation.

11. Improving RAG Quality

- Choose suitable chunk sizes and overlap.
- Use hybrid search where appropriate.
- Apply metadata and access-control filters.
- Use reranking for difficult queries.
- Rewrite ambiguous queries.
- Return source references.
- Evaluate retrieval and answer faithfulness separately.
- Monitor failed and low-confidence queries.

12. Evaluation Metrics

Area	Example Measures
Retrieval	Recall@K, Precision@K, passage relevance
Generation	Correctness, relevance, completeness
Grounding	Faithfulness, citation accuracy
System	Latency, cost, failure rate, user satisfaction

13. Proposed Novel Application: Local Heritage Memory RAG

A promising emerging application is a Local Heritage Memory RAG system for preserving and searching the cultural memory of neighborhoods and small communities. It could combine digitized local newspapers, oral-history transcripts, historical maps, public records, photographs with descriptions and verified community archives.

Users could ask questions such as how a neighborhood changed over decades or which landmarks existed at a particular time. The system would retrieve historical evidence and generate an answer with dates, sources and uncertainty notes.

- Preserves fragmented local knowledge.
- Connects documents with places, people and dates.
- Supports schools, museums and researchers.
- Distinguishes documented facts from unverified stories.
- Provides source references for generated answers.

14. Proposed System Design

- Collect authorized historical and community sources.
- OCR scanned documents and transcribe audio interviews.
- Attach metadata such as date, location and source reliability.
- Create searchable text chunks and embeddings.
- Use hybrid retrieval and metadata filtering.
- Rerank evidence for each question.
- Generate answers with citations.
- Display uncertainty when evidence conflicts or is incomplete.

15. Ethical and Privacy Considerations

RAG systems should respect copyright, privacy, access permissions and data ownership. Sensitive documents require authentication and authorization. Community archives should obtain appropriate consent for personal stories and recordings. Disputed or uncertain claims should not be presented as established facts.

16. Conclusion

RAG connects generative AI with external knowledge through retrieval and contextual generation. Keyword search, vector search, hybrid retrieval, metadata filtering and reranking influence the quality of evidence supplied to the model. Real-world applications include customer support, enterprise knowledge search and research assistance. Local Heritage Memory RAG is a promising application that combines these capabilities

with source attribution and responsible data governance.

17. Submission Information

Report File: RAG_Systems_Research_and_Application_Report.pdf

GitHub Repository: Paste your practical RAG project repository link here.